



Tarea No Presencial correspondiente a la sesión 11 (Grupos de Albizu)

Añade los siguientes métodos a la clase:

1. **oldestPassenger** (). Devuelve el pasajero de más edad sentado en el avión o null si el avión está vacío. Implementa las pruebas de este método.
2. **numberOfFreeSeats** (int row). Devuelve el número de asientos libres en la fila del avión que se le pasa como parámetro. Implementa las pruebas de este método.
3. **print** (). Modifica el método *print* para que presente la separación entre asientos de primera y estándar y el hueco del pasillo. (considerando que el avión tiene tres asientos a cada lado). Utiliza un vector constante {'A','B','C','D','E','F'} para mostrar las letras correspondientes a las columnas. Incluye una X para un adulto, una C, para un niño, un ? para un asiento libre con el siguiente formato:

	A	B	C	D	E	F
0	X	X	?	?	?	X
1	X	X	X	?	?	?
2	?	X	?	?	?	?
3	X	X	?	X	X	X

4	X	X	X	X	X	X
5	X	X	?	X	X	X
6	X	X	X	X	X	X
7	X	X	?	?	?	X
8	?	X	?	?	?	?
9	X	X	?	X	X	X
10	X	X	X	X	X	X
11	X	X	?	?	?	X

4. Implementa un método **getNumPax** que devuelva el número de pasajeros sentados en un área del avión. Un área del avión puede ser primera clase, clase estándar o todo el avión.

```
public int getNumPax (byte area)
```

El parámetro area puede ser un valor 0, 1 ó 2 que se asocia con las constantes:

```
public static final int FIRST_CLASS = 0;  
public static final int STANDARD_CLASS = 1;  
public static final int ALL_CLASS = 2;
```

4. Implementa un método **getNumPaxBySection** *que* devuelva el número de pasajeros sentados en una sección del avión. Una sección se representa mediante la primera posición de una fila y la última posición de una fila. Por ejemplo, la sección (0,0)–(3,5) abarca las filas 0, 1, 2 y 3 completas. La sección (1,1)–(5,4) abarca las posiciones (1,1)(1,2)(1,3)(1,4) de la fila 1, (2,1)(2,2)(2,3)(2,4) de la fila 2 ...

```
public int getNumPax (int row1, int colum1, int row2, colum2)
```

5. Implementa un método **loadPax** para sentar en el avión a un número dado de pasajeros. Se irán sentando en los asientos libres desde las primeras filas del avión.

```
public void loadPax (int paxNumber)
```

Desafío: Proyecto Auditorio

Se desea modelar una parte del auditorio de Oviedo para llevar la gestión de los conciertos que se van celebrando.

Implementar la clase **Concert** (Concierto) con las siguientes propiedades:

- **name**. El nombre del concierto.
- **price**. El precio de la butaca.
- **date**. Fecha en la que se celebra el concierto. Será un objeto de la clase **Date** con tres propiedades: **day**, **month**, **year** y un constructor con tres parámetros. Esta clase **Date** se debe implementar, no se importa de la biblioteca de Java.

Implementar la clase **Auditorium** con las siguientes propiedades:

- **concerts**. Un **ArrayList** para coleccionar la lista de conciertos que se van a celebrar.

Y los siguientes métodos:

6. **seekConcert** (String concertName). Busca un concierto en la colección por el nombre. Devuelve el concierto si existe o null en caso contrario.
7. **addConcert** (String concertName, double price, Date date). Añade un concierto a la colección (la lista de conciertos del auditorio) siempre que no haya otro en la misma fecha.
8. **removeConcerts** (int year). Borrar todos los conciertos de la colección que se hayan celebrado en el año indicado en el parámetro.

Nota: para borrar todos los conciertos se debe elegir:

- a. Un bucle while (incrementando el índice si no está el elemento en esa posición)
- b. Un bucle for (decrementando el índice si se borra el elemento en esa posición).
- c. Un bucle Iterador

No funciona con un **for-each** (ya que cada vez que se borra, cambia el tamaño de la colección).

Añadir una clase de prueba unitaria para probar el correcto funcionamiento de los tres métodos anteriores.